

ORMs & migrations

COMPLETE LEARNING GUIDE

What You'll Learn

An Object-Relational Mapper (ORM) lets application code work with database rows as ordinary objects instead of writing raw SQL by hand, trading some control and performance transparency for a large boost in day-to-day productivity and type safety. Migrations are the companion discipline that makes schema changes trackable, reviewable, and repeatable — a version-controlled, ordered sequence of scripts that take a database from one known schema state to the next, run identically across every developer's machine, staging, and production. This guide covers what an ORM actually does under the hood, walks through defining models and writing migrations with two of the most widely used tools (Prisma and Sequelize, both Node.js ecosystem ORM), and covers the pitfalls that come from trusting an ORM's convenience without understanding the SQL it generates.

1. What an ORM Actually Does

At its core, an ORM maps a class or schema definition in application code to a database table, an instance of that class to a row, and provides methods to query, create, update, and delete rows using the host language's syntax instead of raw SQL strings. This buys real productivity: autocomplete on field names, type-checking, protection against SQL injection by default (parameters are escaped automatically), and a layer of abstraction that can, in principle, be swapped between database engines. The tradeoff is a layer of indirection between the code you write and the SQL that actually executes — code that looks simple can quietly generate an expensive query, which is why understanding the generated SQL (not just trusting the ORM) remains an essential skill.

2. Defining a Model: Prisma Schema

Prisma uses a dedicated schema file (`schema.prisma`) as the single source of truth for both the database schema and the generated, type-safe client code the application imports. A model block declares a table's fields, types, and relationships declaratively; Prisma then generates the actual SQL migrations and a fully typed client from this one file, so the schema definition and the query API a developer uses are always in sync by construction.

PRISMA

```
model Customer {
  id          Int          @id @default(autoincrement())
  email       String       @unique
  fullName    String
  orders      Order[]
  createdAt   DateTime     @default(now())
}

model Order {
  id          Int          @id @default(autoincrement())
  customer    Customer     @relation(fields: [customerId], references: [id])
  customerId  Int
  status      String       @default("pending")
  placedAt    DateTime     @default(now())
}
```

3. Why Migrations Exist: Schema Changes as Version-Controlled Code

Without migrations, schema changes happen by someone manually running an ALTER TABLE statement against a database — a process with no history, no review process, and no guarantee that development, staging, and production ever end up with the exact same schema. A migration is a small, ordered, checked-into-version-control script (each one typically has an 'up' operation that applies the change and a 'down' operation that reverses it) that a migration tool applies in sequence, tracking which migrations have already run in a metadata table so the same migration is never applied twice to the same database.

- Migrations are ordered and checked into version control alongside application code, reviewed in the same pull requests.
- Each environment (dev, staging, production) tracks which migrations it has applied, in a dedicated migrations table.
- A working 'down' migration (to reverse a change) is what makes rollback possible if a deployed migration causes a problem.
- Manually editing a production schema outside of a migration causes 'schema drift' that migration tools can't reconcile automatically.

4. Running Migrations with Prisma

Prisma's migration workflow generates a new SQL migration file automatically by diffing the current schema.prisma against the database's last known state, then applies it. ``migrate dev`` is meant for local development — it can reset the dev database if needed to keep migration history consistent — while ``migrate deploy`` is the production-safe command that only applies pending migrations without ever resetting or altering data destructively, which is why the two commands must never be used interchangeably.

BASH

```
# Local development: generates and applies a new migration, may prompt to reset
npx prisma migrate dev --name add_orders_table
```

```
# Production/CI: applies only pending migrations, never resets data
npx prisma migrate deploy
```

5. Writing a Migration by Hand with Sequelize

Sequelize takes a more explicit, hand-written approach: a migration file exports an `up` function (what to do) and a `down` function (how to undo it), both using the `queryInterface` API to describe schema changes independent of the specific SQL dialect underneath. This gives more manual control than Prisma's auto-diffing approach, at the cost of writing more boilerplate for each change.

JAVASCRIPT

```
module.exports = {
  up: async (queryInterface, Sequelize) => {
    await queryInterface.createTable("orders", {
      id: { type: Sequelize.INTEGER, primaryKey: true, autoIncrement: true },
      customerId: {
        type: Sequelize.INTEGER,
        allowNull: false,
        references: { model: "customers", key: "id" },
      },
      status: { type: Sequelize.STRING, defaultValue: "pending" },
      createdAt: Sequelize.DATE,
      updatedAt: Sequelize.DATE,
    });
  },
  down: async (queryInterface) => {
    await queryInterface.dropTable("orders");
  },
};
```

6. The N+1 Problem, Revisited Through an ORM's Convenience

ORMs make it dangerously easy to trigger the N+1 query problem, because accessing a related object (`order.customer`) often looks like a plain property read in code, hiding the fact that it may silently issue a brand-new database query the instant it's accessed inside a loop. Both Prisma (`include`) and Sequelize (`include`) support eager loading — fetching related data up front in the same query (or a small fixed number of queries) instead of lazily, one at a time — and reaching for that explicitly whenever a loop touches a relation is one of the highest-value ORM habits to build.

7. The Raw SQL Escape Hatch

No ORM's query builder covers every possible query efficiently — complex aggregations, window functions, or a query the ORM would translate into needlessly inefficient SQL are all cases where dropping down to raw SQL (Prisma's `$queryRaw`, Sequelize's `sequelize.query`) is the right, deliberate choice rather than a failure to use the ORM 'properly.' The discipline that matters is using parameterized raw queries (never string-concatenating user input directly into SQL) to preserve the same protection against SQL injection the ORM would have given automatically.

Common Pitfalls

- Editing a production database schema by hand instead of through a migration, causing schema drift that's invisible to the migration tool and hard to diagnose later.
- Never writing (or testing) a working 'down' migration, discovering only during an actual incident that a bad migration can't be rolled back.
- Triggering N+1 queries by accessing a lazy-loaded relation inside a loop, without realizing each access is a separate database round trip.
- Running `migrate dev` (a development command that can reset the database) against a staging or production environment.
- Assuming ORM-generated queries are always efficient without ever inspecting the actual SQL being sent to the database.
- Keeping migration files out of version control, or applying them inconsistently across developers' machines, leading to environments with subtly different schemas.

Key Takeaways

- An ORM trades some control and query transparency for productivity, type safety, and built-in SQL injection protection — understanding its generated SQL is still essential.
- Migrations turn schema changes into version-controlled, ordered, reviewable code instead of untracked manual `ALTER TABLE` statements.
- Every migration should have a working, tested 'down' path, or rollback becomes impossible during an incident.
- Prisma auto-generates migrations by diffing a schema file; Sequelize migrations are written by hand with explicit up/down functions — both track applied migrations in a metadata table.
- Eager-load relations explicitly (include/join) whenever a loop touches related data, to avoid the N+1 query problem ORMs make easy to introduce accidentally.
- Raw, parameterized SQL is a legitimate escape hatch for queries the ORM can't express efficiently — never string-concatenate user input into it.

Learn More

Prisma Documentation — Migrations

<https://www.prisma.io/docs/orm/prisma-migrate>

Sequelize Documentation — Migrations

<https://sequelize.org/docs/v6/other-topics/migrations/>

MDN — SQL Injection Prevention

https://developer.mozilla.org/en-US/docs/Web/Security/Attacks/SQL_Injection